Molikule

Server Deployment Guide

Go / React / PostgreSQL · Company LAN

Internal use only

Covers: build, transfer, database setup, data migration, systemd service

Contents

1	Overview & architecture	3
2	Prerequisites	3
3	Phase 1 — Build on dev machine	3
4	Phase 2 — Copy files to server	4
5	Phase 3 — PostgreSQL setup	4
6	Phase 4 — Data migration	5
7	Phase 5 — Configure the app	5
8	Phase 6 — Test run	6
9	Phase 7 — Systemd service	6
10	Updating the app	7
11	Troubleshooting	7

1. Overview & architecture

Molikule is a supply chain analytics dashboard for internal LAN use. It consists of a Go/Fiber REST API backend, a React/TypeScript/Vite frontend SPA, and a PostgreSQL database pre-populated from Excel data. There is no internet dependency at runtime — all traffic stays on the company network.

Component	Technology	Notes
Backend	Go 1.24+ / Fiber	Compiles to a single Linux binary
Frontend	React / TypeScript / Vite	Compiled to static files in dist/
Database	PostgreSQL 14+	Populated once via Python script
Auth	JWT (8-hour tokens)	No refresh token — re-login per session

Request flow on the LAN:

```
Employee browser  →  http://server-ip:8080
                              ↓
                     Go binary (port 8080)
```

2. Prerequisites

Tool	Version	Where needed
Go	1.24+	Dev machine (build only)
Node.js	18+	Dev machine (build only)
PostgreSQL	14+	Server
Python	3.8+	Server (migration only)
SSH / SCP	any	Dev machine (file transfer)

Go and Node are only needed on your dev machine to build. The server only needs PostgreSQL and Python — nothing else.

3. Phase 1 — Build on your dev machine

Run these on your laptop / workstation — not on the server

Build the frontend

This compiles all React/TypeScript code into plain HTML, CSS and JS files that any browser can read. No Node.js is needed on the server — just these files.

```
cd frontend
npm install
npm run build
# Output: frontend/dist/
```

Build the backend binary

Go compiles your entire application — runtime, dependencies, all handlers — into a single executable file. The GOOS and GOARCH flags tell Go to target Linux even if you are building on macOS or Windows.

Tip: you can embed frontend/dist/ directly into the binary using go:embed — then you only need to copy one file to the server. See the README

```
GOOS=linux GOARCH=amd64 go build -o molikule ./cmd/server/

# If your dev machine is already Linux:
go build -o molikule ./cmd/server/

# Output: backend/molikule  (a single executable file)
```

4. Phase 2 — Copy files to the server

Run these from your dev machine terminal

Use SCP (secure copy) to transfer the compiled binary and frontend files to the server. Replace **user** and **server-ip** with your actual credentials.

```
# Create the directory on the server first
ssh user@server-ip 'sudo mkdir -p /opt/molikule/frontend'

# Copy the Go binary
scp backend/molikule user@server-ip:/opt/molikule/

# Copy the compiled React app
scp -r frontend/dist user@server-ip:/opt/molikule/frontend/

# If you have an Excel file to migrate, copy it too
scp purchase_records.xlsx user@server-ip:/opt/molikule/datamigration/files/
```

Expected layout on the server after this step:

```
/opt/molikule/
  molikule              <- Go binary (the whole app)
  frontend/
    dist/               <- compiled React SPA
      index.html
      assets/
```

5. Phase 3 — PostgreSQL setup

SSH into the server and run these once

Install PostgreSQL

```
sudo apt update
sudo apt install postgresql -y

sudo systemctl enable postgresql
sudo systemctl start postgresql
```

Create the database and user

```
-- Inside the psql shell:
CREATE DATABASE molikule;
CREATE USER molikule_user WITH PASSWORD 'your-strong-password';
GRANT ALL PRIVILEGES ON DATABASE molikule TO molikule_user;
\q
```

Apply the schema migration

This creates all the tables the app needs. Run it from the repo directory you cloned or transferred to the server.

```
# From the molikule repo directory on the server
psql postgres://molikule_user:your-strong-password@localhost:5432/molikule \
  -f backend/sql/migrations/001_initial.sql

# Verify tables were created
psql postgres://molikule_user:your-strong-password@localhost:5432/molikule \
  -c '\dt'
```

6. Phase 4 — Data migration (Python script)

Install Python dependencies

```
pip install pandas openpyxl psycopg2-binary python-dotenv --break-system-packages
```

The migration script (datamigration/migrate.py)

```
import os, pandas as pd, psycopg2
from psycopg2.extras import execute_batch
```

Run the script

ON CONFLICT DO NOTHING makes the script safe to re-run. If you ever need to reload data after a DB wipe, just run it again.

```
DATABASE_URL = os.getenv('DATABASE_URL')
EXCEL_FILE   = 'files/purchase_records.xlsx'

df = pd.read_excel(EXCEL_FILE)
df.columns = [c.strip().lower().replace(' ', '_') for c in df.columns]
df.dropna(how='all', inplace=True)

conn = psycopg2.connect(DATABASE_URL)
cur  = conn.cursor()

INSERT_SQL = '''
    INSERT INTO purchase_records (
        order_id, supplier, item_name, quantity, unit_price, order_date, status
    ) VALUES (
        %(order_id)s, %(supplier)s, %(item_name)s,
        %(quantity)s, %(unit_price)s, %(order_date)s, %(status)s
    ) ON CONFLICT (order_id) DO NOTHING;
'''

execute_batch(cur, INSERT_SQL, df.to_dict(orient='records'), page_size=500)
conn.commit()
cur.close()
conn.close()
print(f'Done — {len(df)} rows imported.')
```

```
# Verify row count
psql $DATABASE_URL -c 'SELECT COUNT(*) FROM purchase_records;'
```

7. Phase 5 — Configure the app

Create the .env file next to the binary

```
nano /opt/molikule/.env
```

.env file contents:

Variable	Description	Example
DATABASE_URL	Full PostgreSQL connection string	postgres://user:pass@localhost/molikule
JWT_SECRET	Secret for signing auth tokens. Min 32 chars.	openssl rand -hex 32
PORT	Port the HTTP server listens on	8080

> Generate a strong JWT secret with: openssl rand -hex 32

8. Phase 6 — Test run

Run manually first to confirm everything works before setting up the service

```
# Make binary executable
chmod +x /opt/molikule/molikule

# Run it
cd /opt/molikule
./molikule
```

While it is running, open a browser on any machine on the LAN and go to **http://server-ip:8080**. You should see the Molikule login page. If it works, press Ctrl+C to stop it — you will run it properly as a service next.

If the page does not load:

Check	Command
Binary is running	`ps aux | grep molikule`
Port 8080 is open	`ss -tlnp | grep 8080`
Firewall allows port 8080	`sudo ufw allow 8080`
PostgreSQL is accepting connections	`psql $DATABASE_URL -c 'SELECT 1'`
.env values are correct	`cat /opt/molikule/.env`

9. Phase 7 — Systemd service

Runs the app permanently in the background, survives reboots

Create the service file

```
sudo nano /etc/systemd/system/molikule.service
```

Paste this content:

```
[Unit]
Description=Molikule Supply Chain Dashboard
```

Enable and start the service

Directive	What it does
`After=postgresql.service`	Waits for Postgres to be ready before starting
`EnvironmentFile`	Loads DATABASE_URL, JWT_SECRET, PORT from .env
`Restart=always`	Automatically restarts if the process crashes
`RestartSec=5`	Waits 5 seconds before each restart attempt
`User=www-data`	Runs as a low-privilege user for security

```
[Install]
WantedBy=multi-user.target
```

10. Updating the app

When you make code changes, the update process is just two commands run from your dev machine:

> The database never needs to be touched again after the initial migration. Only rebuild and copy the parts that changed.

```
# 2. Copy new binary to server and restart
scp backend/molikule user@server-ip:/opt/molikule/
ssh user@server-ip 'sudo systemctl restart molikule'
```

11. Troubleshooting

Symptom	Likely cause	Fix
Binary exits immediately	Bad .env / DB unreachable	`journalctl -u molikule -n 50`
Login page loads, login fails	Wrong JWT_SECRET or DB data	`Check .env JWT_SECRET`
'exec format error'	Binary built for wrong OS	`Rebuild with GOOS=linux`
Port 8080 refused from LAN	Firewall blocking port	`sudo ufw allow 8080`
`DB connection refused`	Postgres not running	`sudo systemctl start postgresql`
Migration script KeyError	Excel column name mismatch	`Add df.rename() in migrate.py`
Service won't start after reboot	Not enabled	`sudo systemctl enable molikule`

Useful log commands:

```
# Live service logs
sudo journalctl -u molikule -f

# Last 100 lines
sudo journalctl -u molikule -n 100

# Check all listening ports
ss -tlnp
```